Lab1 实验报告

实验要求

实现一个五级流水线 CPU，支持 64 位算术运算。

支持以下指令：

- `addi, xori, ori, andi`
- `add, sub, and, or, xor`
- `addiw, addw, subw`

实验过程

这一小节中，所有 AI 帮助完成的部分都进行了标注

参考

- 《自己动手写 CPU》。雷思磊著。北京：电子工业出版社，2014 年。
- AI（iflow, GLM-4.7）。
- 来自 2024 级工科试验班（现微纳电子与集成电路创新学院）周吴凡同学实现的完整五级流水线 CPU。

Step 0

确定了项目结构如下

```
/vsrc/
|---- /include/
|      |---- /common.sv
|      |---- /config.sv
|      |---- /pipeline_def.sv        // 定义了常用常量与流水线寄存器结构体
|
|---- /src/
|      |---- /components/            // 包含了基础组件的实现
|      |      |---- /alu.sv
|      |      |---- /brach_unit.sv    // 未使用
|      |      |---- /csr.sv           // 未使用
|      |      |---- /imm_gen.sv       // 立即数生成器（提供扩展功能）
|      |      |---- /regfile.sv
|      |
```

```
|      |---- /control/
|      |      |---- /forward_unit.sv  // 数据转发控制单元
|      |
|      |---- /pipeline/
|      |      |---- /if_stage.sv
|      |      |---- /id_stage.sv
|      |      |---- /ex_stage.sv
|      |      |---- /mem_stage.sv
|      |      |---- /wb_stage.sv
|      |      |---- /pipeline_top.sv  // 流水线顶层
|      |
|      |---- /core.sv
|
|---- /util/
|
|---- /SimTop.sv
```

Step 1

参考《自己动手写 CPU》，完成了 `if_stage.sv` 及相关组件、流水线寄存器。但在参考完成 `id_stage` 时，发现书籍与实验要求相去甚远，自行修改无果后，求助 AI 帮我完成了剩余部分。

Step 2

调试阶段。将在接下来选取调试时发现的几个代表性问题记录。

调试阶段

这一小节中，所有 AI 帮助完成的部分都进行了标注

*带 * 的问题是在调试阶段遇到的比较有挑战性的问题*

问题 1

由于在取指阶段，两条指令之间相隔 3 个时钟周期，而在提交时，`valid` 信号在 `reg_write` 信号（即写回阶段寄存器的写使能信号）为真时即为真，导致会在连续多个时钟周期内提交同一条指令（这部分逻辑由 AI 实现）。

解决 1

修改 `valid` 信号，使其在检测到 `pc` 发生变化，且 `reg_write` 信号为真时，才产生提交。

```
u64 prev_pc;
```

```
2
3  always_ff @(posedge clk) begin
4      if (mem_wb_reg.reg_write) begin
5          if (mem_wb_reg.pc != prev_pc) begin
6              commit_valid <= `Enable;
7              prev_pc <= mem_wb_reg.pc;
8          end else begin
9              commit_valid <= `Disable;
10         end
11     end else begin
12         commit_valid <= `Disable;
13     end
14 end
```

问题 2

通过分析测试时的错误指令发现，CPU 在处理一条指令时，错误地写入了 `x31` 寄存器。经检查，发现我在代码中将常量 `RegNone`（在不指定寄存器时的默认值，应当被认为是一个不存在的寄存器）定义为了 `5'b11111`，即 `x31` 寄存器，导致了错误的写入 。

解决 2

将 `RegNone` 修改成了 `5'b00000`。

问题 3

通过分析测试时的错误指令发现，存在一条指令，其目的寄存器为 `x0`。尽管 AI 的实现中，实现了当 `waddr`（被写入的寄存器）为 `x0` 时，不进行数据写入的逻辑，但在暴露寄存器接口时，为了同步最新的值，添加了如下判断逻辑：

```
1  end else if ((i == {27'b0, waddr}) && (we == `Enable)) begin
2      gpr_out[i] = wdata;
```

导致暴露的接口中，`x0` 寄存器依然被错误地修改了。

解决 3

在暴露接口时添加逻辑，使 `x0` 寄存器强制性传递 `ZeroWord`（被定义为 `64'b0`）。

***问题 4**

通过分析测试时的错误指令发现，由于一条指令需要持续存在 3 个时钟周期，当遇到源寄存器和目的寄存器相同的情况时，`alu` 会多次读取进入到访存、写回阶段的结果（由于数据转发机制），从而将一条指令反复执行，计算出错误的结果（这部分逻辑为 AI 实现）。

解决 4

在这里，AI 给出了一个错误但可以通过测试的方法。

AI 直接在源寄存器与目的寄存器相同时，阻断了数据转发。虽然达到了解决问题的目的，但这导致如果源寄存器在一开始确实发生了数据冒险，将无法获得正确的源操作数。

在测试文件中可能没有类似的指令，导致这种修改可以通过测试。

到这里，代码实际上已经可以通过全部测试。

```
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000080010004
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 2,408 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 16,385
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 6,804,401 instr/s
```

但在 **解决 4** 中，依然有一个遗留的问题。如果不解决这个错误的逻辑，很可能影响未来功能的扩展。

***问题 5**

修复 **解决 4** 产生的错误逻辑。

解决 5

为 `ex_stage` 也加入了 `step` 信号。当 `step` 信号为假时，阻塞 `ex_stage` 的信号，不让它传入到后续的阶段当中，这样，`ex_stage` 就不会用自己计算出来的结果反复计算了。

后续在引入多周期计算时，可以通过控制 `step` 信号（实际上是控制 `execute_ok` 信号）实现对信号传输的控制。

到此，终于完成了一个没有设计错误、方便未来扩展（大概）的五级流水线 CPU，最后也成功通过了测试。

```
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000080010004
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 2,165 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 16,385
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 7,568,129 instr/s
```

同时可以看到，CPU 的性能也获得了提升。

实验结果

最终，得到了一个实现了 Lab1 基础要求的五级流水线 CPU。通过整个实验，我发现：

1. 仅仅依靠 AI（或者说，依靠我使用的这款 AI，具体结论可能随着不同的 AI 类型，可以调度的算力等不同）无法正确完成 CPU 的代码工作；
2. 流水线 CPU 的实现难点不在于各个组件、流水线逻辑的实现，而在于时序逻辑的处理。如何在时间层面上调度数据是实现流水线 CPU 的难点。